

KULLIYAH OF ENGINEERING
MECHATRONICS SYSTEM INTEGRATION (MCTA 3203)
SEMESTER 2, 2025 / 2026

WEEK 9: BLUETOOTH DATA INTERFACING

SECTION 2

GROUP 4

INSTRUCTORS : ZULKIFLI BIN ZAINAL ABIDIN & WAHJU SEDIONO

Date of Submission: 13th May 2026

NO	NAME	MATRIC NUMBER
1	Hannah Irdeena Binti Azrulhafiz	2410018
2	Azzam Azhari Noureldein Elamin	1827295
3	Kamarul Zafri Bin Kamarul Ariffin	2323189

ABSTRACT

This module investigates the integration of wireless communication in a mechatronic system using the ESP32 development board utilising the bluetooth technology with a paired device such as a smartphone. Focusing on establishing a smooth and functional wireless bridge between the ESP 32 and a smartphone to monitor sensor data and execute the commands. Utilizing the Frequency Hopping Spread Spectrum (FHSS) over a 2.4 GHz, the system successfully transmits analog values from the potentiometer to the mobile terminal app. In addition, the system demonstrates bidirectional communication by controlling the LED ON and LED OFF and servo motor through specific commands (e,g 'O' and 'F') sent from a smartphone.

This setup shows how wireless communication is executed with various simple hardware combined into a single user friendly system. Overall, the experiment successfully achieves its objectives and provides a foundation for IoT based automation and smart monitoring applications.

TABLE OF CONTENTS

NO.	TOPIC	PAGE
1.	INTRODUCTION	4
2.	MATERIALS AND EQUIPMENT	5
3.	EXPERIMENTAL SETUP	6
4.	METHODOLOGY	7
5.	DATA COLLECTION	10
6.	DATA ANALYSIS	11
7.	RESULTS	12
8.	DISCUSSION	12
9.	CONCLUSION	13
10.	RECOMMENDATIONS	14
11.	REFERENCES	14
12.	APPENDICES	15

1. INTRODUCTION

The experiment's purpose is to understand and implement wireless serial communication using Bluetooth, specifically utilising the ESP32's native Bluetooth capabilities to create a remote monitoring and control system. Bluetooth provides a short range, low power wireless standard ideal for connecting mechatronic systems to mobile user interfaces without complex network infrastructure.

The ESP32 facilitates Bluetooth communication by utilising a 2.4 GHz radio frequency to transmit data packets through Frequency Hopping Spread Spectrum (FHSS), preventing interference by switching between 79 channels. To connect to a phone, the ESP32 employs the Bluetooth Classic Serial Port Profile (SPP), creating a virtual wireless bridge that mimics a physical wired UART connection. Once paired, the smartphone treats the Bluetooth stream as a standard serial interface, allowing simple `SerialBT.print()` commands to send and receive data.

In the context of the Internet of Things (IoT), Bluetooth acts as a "last mile" connectivity solution. A microcontroller reads local sensor data and sends it via Bluetooth to a smartphone, which then acts as a gateway to forward data to cloud servers via Wi Fi. For this task, the system was expanded beyond basic LED control to incorporate a potentiometer for continuous sensor data transmission and a servo motor for physical actuation based on remote commands. It is expected that the ESP32 will reliably stream sensor values while instantly reacting to received characters to adjust the servo and LED states.

2. MATERIALS AND EQUIPMENT

1 x ESP32 development board

1 x Potentiometer (10k Ω)

1 x Servo motor (SG90)

1 x Smartphone (Android/iOS)

Jumper wires and Breadboard

Micro-USB cable for ESP32 power/s programming

Software: Arduino IDE, "Serial Bluetooth Terminal" app (by Kai Morich)

3. EXPERIMENTAL SETUP

Hardware Setup

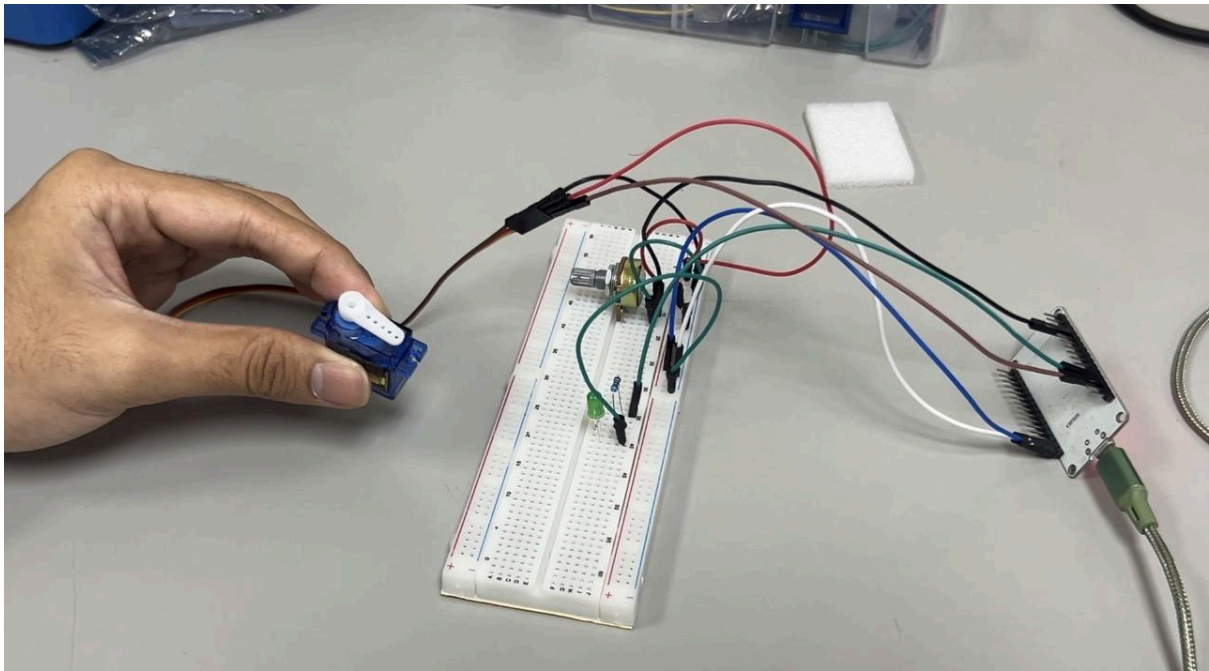


Figure 1 shows the component and its connection.

1. The ESP32 development board serves as the central processing unit, utilizing its internal 2.4 Ghz radio for bluetooth communication.
2. A potentiometer is connected to GPIO 34 on the ESP32 acting as the channel to capture variable analog signals.
3. The servo motor is integrated on the ESP by pin GPIO 13.
4. An external LED is connected in series with a resistor and is connected to the board by GPIO 12 pin.
5. All components are mounted on a common breadboard with the power and GND supplied to the ESP32 via jumper wires.
6. A smartphone runs the Serial Bluetooth Terminal acting as a remote that allows character-based command entry and real time monitoring of the potentiometer value.

Software Setup

1. The Bluetooth serial device was initialised with “Group_04” to allow the smartphone to discover and pair with the board.
2. The Arduino IDE Serial Monitor was set to a baud rate of 9600 to observe debug messages during transmission.

4. METHODOLOGY

4.1 Task 1 Modification

To fulfill the task of integrating a sensor and a servo, the ESP32Servo library was incorporated. A servo object was attached to GPIO 13. The switch case logic was designed to handle five distinct commands: 'O' and 'F' to control the GPIO 12 LED, and 'S' and 'R' to set the servo to 90° and 0° respectively.

4.2 Control Protocol Design

ESP32 → Smartphone (Telemetry):

The ESP32 reads the analog value (0–4095) from the potentiometer every 1 second and transmits a formatted string:

"Sensor Value: [value]\n"

Smartphone → ESP32 (Commands):

The smartphone sends single-byte ASCII characters:

'O' → Turn LED ON (Returns "LED is ON")

'F' → Turn LED OFF (Returns "LED is OFF")

'S' → Move Servo to 90° (Returns "Servo moved to 90")

'R' → Move Servo to 0° (Returns "Servo reset to 0")

4.3 Arduino Code (Task 1 Implementation)

```
#include "BluetoothSerial.h"
```

```
#include <ESP32Servo.h>
```

```
BluetoothSerial SerialBT;
```

```
Servo myServo;
```

```
// Hardware Pins
```

```
const int potpin = 34; // Potentiometer on GPIO 34
```

```
const int LED = 12; // LED on GPIO 12
```

```
const int servoPin = 13; // Servo Signal on GPIO 13
```

```
// Variables
```

```
int potValue = 0;
```

```
char command;
```

```
void setup() {
```

```
    Serial.begin(9600);
```

```
    // --- THIS IS YOUR BLUETOOTH SSID ---
```

```
    SerialBT.begin("Group_4");
```

```
    pinMode(LED, OUTPUT);
```

```
    myServo.attach(servoPin);
```

```
    myServo.write(0);
```

```
    Serial.println("Bluetooth initialized... Ready to receive commands.");
```

```
}
```

```
void loop() {
```

```
    // 1. Read the Potentiometer (Sensor)
```

```
    potValue = analogRead(potpin);
```

```
    // 2. Send Potentiometer value to Smartphone Terminal
```

```
    SerialBT.print("Sensor Value: ");
```

```
SerialBT.println(potValue);

// 3. Check for Bluetooth Commands
if (SerialBT.available()) {
    command = SerialBT.read();

    Serial.print("Command received: ");
    Serial.println(command);

    // Control LED and Servo based on commands
    switch (command) {
        case 'O': // Turn LED ON
            digitalWrite(LED, HIGH);
            SerialBT.println("LED is ON");
            break;
        case 'F': // Turn LED OFF
            digitalWrite(LED, LOW);
            SerialBT.println("LED is OFF");
            break;
        case 'S': // Move Servo to 90 degrees
            myServo.write(90);
            SerialBT.println("Servo moved to 90");
            break;
        case 'R': // Reset Servo to 0 degrees
            myServo.write(0);
```

```

        SerialBT.println("Servo reset to 0");

        break;

    }

}

delay(1000); // Send data every second

}

```

4.4 Troubleshooting

Problem	Possible Cause	Solution
App cannot find ESP32	Bluetooth not initialised	Check Serial Monitor for "Bluetooth initialized"
Analog read always 0	Floating pin	Verify wiper connection to GPIO 34

5. DATA COLLECTION

LED Control Commands (Smartphone to ESP32):

Command Sent	Expected App Feedback	Observed Hardware State	Response Timing
None	-	OFF	-
O	LED is ON	ON	Instant
F	LED is OFF	OFF	Instant

Servo Motor Actuation Data (Smartphone to ESP32):

Command	Expected App	Observed	System Stability
---------	--------------	----------	------------------

Sequence	Feedback	Hardware Action	
S	Servo moved to 90	Servo immediately moved to 90°	Stable
R	Servo reset to 0	Servo immediately moved to 0°	Stable
Alternating 'S' and 'R'	Servo moved to 90 Servo reset to 0 (repeating)	Continuous back and forth movement	No freezing

6. DATA ANALYSIS

During the example task, the analog data collected from the potentiometer and displayed on IDE's Serial Monitor was carefully analysed to evaluate the accuracy and overall performance of the ESP32's data processing wirelessly. By the numerical readings collected during the task, it was examined to observe the nature of the values when the potentiometer knob was manually adjusted. It could be immediately identified, by the stable readings, when the knob was stationary and moving the numerical values increases and decreases when the physical adjustments were made.

It is also seen in addition to the sensor data, the built in LED was evaluated by comparing its ON and OFF states with the control commands sent from the smartphone via bluetooth. Initially, when there was no commands sent, the LED remained OFF. When the command 'O' was sent via Serial Bluetooth Terminal, the LED turned ON immediately, confirming the correct bluetooth communication and proper common execution by the microcontroller. When the 'F' command was sent, the LED turned OFF. The timing of the response was observed for any noticeable delays between the command transmission and hardware response, proven to be instantaneous.

The performance of the servo motor was also evaluated under manual wireless control. When the command were sent via the smartphone, the servo immediately move to the assigned angle. Consistently sending alternating commands shows the system could handle the movements without freezing. Overall, the experiment successfully demonstrated a reliable closed-loop environment monitor and an actively responsive manual control function.

7. RESULTS

The system successfully achieved all objectives of the task:

- **Wireless Monitoring:** The smartphone app reliably displayed the potentiometer's analog value every 1 second, correctly showing stable values when stationary and progressive changes when rotated.
- **Wireless Control:** Sending single characters from the smartphone successfully controlled the LED (toggling ON/OFF via 'O' and 'F') and positioned the servo motor (90° and 0° via 'S' and 'R').
- **System Stability:** The system handled bidirectional communication seamlessly, maintaining a stable connection without freezing, even when alternating servo commands were sent rapidly.

8. DISCUSSION

Task 1 Discussion: Bluetooth in IoT Applications :

Bluetooth is incorporated into IoT applications primarily as a low power, short range protocol ideal for "edge" devices. In a typical IoT architecture, an ESP32 acts as an edge node. It collects data from local sensors (like the potentiometer) and sends it via Bluetooth to a smartphone. The smartphone acts as a gateway, using its Wi Fi or 4G/5G connection to upload the data to cloud platforms. Conversely, the cloud can send commands down to the smartphone, which relays them via Bluetooth to the ESP32 to trigger actuators (like the servo). This avoids putting the power hungry Wi Fi burden directly on the microcontroller, extending battery life for portable IoT devices.

Sources of Error and Limitations:

1 Second Blocking Delay: The delay(1000) blocks the code from checking for new Bluetooth commands during that window, meaning a command sent right after a sensor transmission could wait up to 1 second to be executed.

Bluetooth Classic Range: Limited to approximately 10 metres indoors. Beyond this, packet loss increases, causing stale sensor data.

No Data Security: Data is transmitted in plain text. Any paired device could intercept values or send commands.

9. CONCLUSION

In conclusion, the task successfully demonstrate the integration integration of bidirectional wireless communication system using the ESP32 microcontroller, sensor and a device. It can be found that the ESP32 is highly effective at managing physical hardware and wireless data via the Bluetooth Classic Serial Port Profile (SPP). From reading analog voltage signals from a potentiometer, immediate toggling ON and OFF of the LED and receiving commands to drive a physical actuator this confirms the accuracy and stability of the system.

The aftermath of this experiments supports the hypothesis of the ESP32 as a reliable, closed-loop wireless bridge to execute real-time commands from a mobile. It seems the utilisation of Frequency-Hopping Spread Spectrum (FHSS) has ensured the data flow is stable and contradict any latency in a 2.4GHz environment. This simple setup has significant implications for a far more broader mechatronics system integration and IoT architectures, such as automated manufacturing hubs and remote monitoring stations.

10. RECOMMENDATIONS

Improvements and Modifications:

1. **Replace Delay with Timers:** Using the `millis()` function for non-blocking timing would allow the system to read the sensor and check for commands simultaneously, eliminating the 1 second command latency.
2. **Integrate DHT22 Sensor:** As suggested in the tutorial theory, replacing the potentiometer with a DHT22 temperature and humidity sensor would provide practical environmental data for a real-world IoT monitoring system.
3. **Implement BLE:** Switching to Bluetooth Low Energy (BLE) using the Nordic UART Service would significantly reduce power consumption compared to Bluetooth Classic SPP.

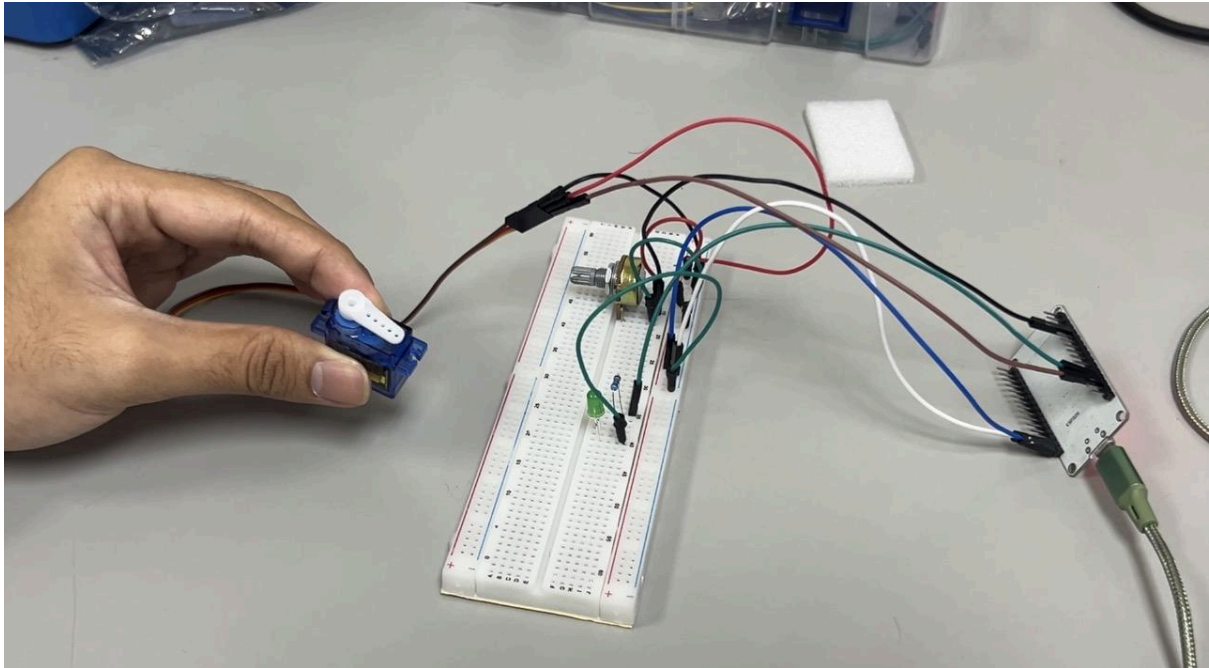
Insights and Lessons:

1. The ESP32's GPIO 34 is an input only pin; care must be taken when selecting pins for actuators, which must be placed on standard GPIO pins (like GPIO 12 and 13).
2. When pairing an ESP32 for the first time, the smartphone may prompt for a PIN. The default PIN for ESP32 Bluetooth Classic is usually "0000" or "1234".

11. REFERENCES

1. HowToMechatronics. Arduino and HC-05 Bluetooth Module Tutorial.
<https://howtomechatronics.com/tutorials/arduino/arduino-and-hc-05-bluetooth-module-tutorial/>
2. Week 9: Bluetooth Data Interfacing (ver. 2). MCTA3203 Mechatronics System Integration.
3. Arduino Project Hub. Basic Bluetooth communication with Arduino & HC-05.
<https://projecthub.arduino.cc/superturis/basic-bluetooth-communication-with-arduino-hc-05-3a431c>

12. APPENDICES



ACKNOWLEDGEMENT

We would like to express our sincere gratitude to Dr. Zulkifli Bin Zainal Abidin and Dr. Wahyu Sediono for their continuous support and for providing the essential resources and facilities required to carry out this project. We also extend our heartfelt appreciation to everyone who contributed to the success of this lab report through their valuable insights, assistance, and feedback.

STUDENTS DECLARATION

We hereby affirm that the work presented in this report is entirely our own. All sources used for background information and theory have been properly cited, and have strictly adhered to the principles of academic integrity throughout this report.